Project 3 - "Forma AI": AI-Augmented Dynamic Form Engine

Domain: InsurTech & Workflow Automation

Problem Statement: Large organizations (Insurance, Healthcare) rely on massive, branching forms (e.g., 50+ questions where Question 12 dictates Questions 13-20). Building these dynamically in React and managing the complex validation state in MongoDB is notoriously difficult. Adding AI to interpret unstructured user input makes it exponentially harder.

Use Case: A user files a complex insurance claim on Forma AI. Instead of filling out 50 strict dropdowns, they type a natural language paragraph: *"I hit a deer on I-95 yesterday in my Honda, and the windshield shattered."* The backend LLM parses this unstructured text, automatically pre-filling the structured React form fields (incidentType: "animal_collision", vehicle: "Honda", damage: "windshield"). The React UI dynamically reveals only the remaining, necessary questions based on a complex JSON schema stored in MongoDB, drastically reducing form friction.

Key Modules:

- **Dynamic Form Renderer (React & React Hook Form):** A frontend engine capable of generating complex, nested UIs dynamically entirely from a backend JSON schema.
- **Schema Store (MongoDB):** Stores the rules, validation logic, and branching paths for hundreds of different form types.
- **LLM Extraction API (Node.js & LangChain):** Uses an LLM (like GPT-4o or a local model) to parse unstructured user text and map it to the strict JSON schema required by the form.
- **State Management (Redux / Zustand):** Manages the complex, deeply nested state of the form as the user interacts with it.

Week-wise Development Plan:

Week	Backend & AI Engine (Node.js, LangChain, MongoDB)	Frontend & Forms (React Hook Form)
Week 1	**Schema Design:** Design a MongoDB schema to represent a dynamic form (Fields, Types, Validation Regex, and "Show If" conditional logic).	**Dynamic Renderer:** Build a React component that fetches the JSON schema and dynamically renders text inputs, dropdowns, and checkboxes.

Week	Backend & AI Engine (Node.js, LangChain, MongoDB)	Frontend & Forms (React Hook Form)
Week 2	**LLM Integration:** Integrate LangChain in Node.js. Write prompts instructing the LLM to extract specific entities from a user's story and output them strictly as JSON.	**AI Input UX:** Build the frontend "Magic Input" box where users type their story, alongside a loading skeleton state while the AI processes.
Mid-Project Review	**Schema Audit:** Prove the backend can serve a complex, 3-level deep branching JSON schema correctly.	**Dynamic Rendering Check:** Validate that selecting "Yes" on Question A dynamically renders Question B based purely on the backend schema rules.
Week 3	**Extraction API:** Connect the LLM output to the React form. When the AI returns JSON, automatically populate the corresponding React Hook Form fields.	**Validation UI:** Implement complex client-side validation. Highlight fields that the AI missed or extracted incorrectly, forcing human review.
Week 4	**Save & Resume:** Implement API routes to save the partially completed form state to MongoDB, allowing users to return later.	**Refine & Polish:** Polish the UI animations (e.g., smoothly expanding new form sections as conditional logic evaluates to true).
Final Review	A highly scalable architecture for managing unstructured data and complex business rules.	A seamless, AI-augmented user experience that modernizes legacy data entry.

Common Features Across All Projects

- **Advanced React Patterns:** Moving past basic useState. Heavy use of custom hooks, complex state management (Zustand/Redux), Canvas rendering, and Dynamic UI generation.

- **Beyond CRUD (Create, Read, Update, Delete):** These architectures utilize WebSockets for real-time sync, Event Sourcing for immutability, and AI pipelines for unstructured data parsing.
- **Scalable Node.js:** Structuring Express APIs using enterprise patterns (CQRS, Service Layers, Dependency Injection) rather than monolithic, tightly coupled routes.
- **Complex Data Modeling:** Designing MongoDB schemas that handle append-only logs, deeply nested JSON trees, and CRDT binary states, proving deep NoSQL mastery.

